

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS				Total Marks: 50
Q.No	Question	Bloom's Level	Marks	
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5	
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5	
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5	
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5	
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5	

6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5
7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct

I MONIKA.B (192571002) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: __MONIKA.B_____

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

Bloom's Level: BL2 – Understand | Marks: 5

Assumptions

- SID = Student ID, SName = Student Name
- CID = Course ID, CName = Course Name, Instructor = who teaches the course
- Grade = the mark/grade a specific student got in a specific course

Step 1 — Functional Dependencies

#	FD	Reasoning
1	SID → SName	Each student ID identifies exactly one student name
2	CID → CName, Instructor	Each course ID identifies exactly one course name and instructor
3	SID, CID → Grade	A grade only makes sense for a particular (student, course) pair

Step 2 — Candidate Key (attribute-closure test)

Compute closure({SID, CID})⁺ under the FDs above:

```
{SID, CID}
SID -> SName    add SName
CID -> CName, Instructor  add CName, Instructor
SID,CID -> Grade    add Grade
closure({SID,CID}) = {SID, CID, SName, CName, Instructor, Grade} = ALL attributes
```

Since {SID, CID}⁺ = all 6 attributes, and neither {SID} nor {CID} alone can reach all attributes, {SID, CID} is minimal → it is the only candidate key.

Tool verification (Python attribute-closure algorithm):

```
>>> closure(["SID","CID"], FDs)
{'SID', 'CID', 'SName', 'CName', 'Instructor', 'Grade'}
>>> candidate_keys(all_attrs, FDs)
[('SID', 'CID')]
```

Result — Candidate Key = {SID, CID} (composite / primary key). SID and CID individually are prime attributes; SName, CName, Instructor, Grade are non-prime.

Q2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Bloom's Level: BL3 – Apply | Marks: 5

Concept

A relation is in 1NF when every attribute holds a single atomic value and there are no repeating groups / multi-valued cells and no nested tables.

Example — Unnormalized relation (UNF)

SID	SName	Courses
S1	Arun	C101, C102
S2	Bala	C103

The Courses column violates 1NF — it stores a multi-valued, non-atomic set of course codes in a single cell (a repeating group).

Step-by-step conversion

- Step 1: Identify the attribute(s) holding multiple values per row — here, Courses.
- Step 2: Remove the repeating group by giving each (student, course) combination its own row instead of packing several course codes into one cell.
- Step 3: Re-check every remaining cell — SID, SName, CID must each now hold exactly one atomic value.
- Step 4: Define the key. A single SID no longer uniquely identifies a row (a student can appear twice), so the key becomes composite: {SID, CID}.

Resulting 1NF relation

SID	SName	CID
S1	Arun	C101
S1	Arun	C102
S2	Bala	C103

Every cell now holds a single atomic value and no cell repeats a group → the relation is in 1NF.

Q3. Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

Bloom's Level: BL3 – Apply | Marks: 5

Step 1 — Find the candidate key

```
closure(A):
  A -> BC    add B,C    {A,B,C}
  BC -> D    add D      {A,B,C,D}
  D -> E     add E      {A,B,C,D,E}
closure(A) = {A,B,C,D,E} = ALL attributes
```

Tool verification (Python):

```
>>> closure(["A"], FDs)
{'A', 'B', 'C', 'D', 'E'}
>>> candidate_keys(attrs, FDs)
[('A',)]
```

Candidate key = {A}. A is a single (non-composite) attribute.

Step 2 — Check for partial dependency

A partial dependency exists only when a non-prime attribute depends on a proper subset of a composite candidate key. Because the only candidate key here, {A}, has just one attribute, it has no proper non-empty subset — so a partial dependency is structurally impossible.

Conclusion: R(A,B,C,D,E) is already in 2NF (1NF + no partial dependency).

Note — what the FD chain actually represents

The chain $A \rightarrow BC \rightarrow D \rightarrow E$ is not a partial dependency, it is a transitive dependency (B,C,D,E depend on the key A, but D and E depend on A only indirectly, through BC and D respectively). Transitive dependency is a 3NF concern, not a 2NF concern, so it is outside the scope of this question — it would be resolved (for completeness) by further splitting into R1(A,B,C), R2(B,C,D), R3(D,E) if 3NF were required.

Q4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

Bloom's Level: BL3 – Apply | Marks: 5

Step 1 — Functional dependencies & key

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{ManagerID}$ (EmpID is the key; closure(EmpID) reaches DeptName too, via DeptID, so $\text{EmpID}^+ = \text{all attributes}$)
- $\text{DeptID} \rightarrow \text{DeptName}$

Candidate key = {EmpID}.

Step 2 — Identify the transitive dependency

$\text{EmpID} \rightarrow \text{DeptID}$ and $\text{DeptID} \rightarrow \text{DeptName}$, so $\text{EmpID} \rightarrow \text{DeptName}$ holds only transitively (through the non-prime attribute DeptID). This violates 3NF, whose rule is: for every FD $X \rightarrow Y$, either X is a superkey, or every attribute in Y is a prime (key) attribute. Here DeptID is not a superkey and DeptName is not prime $\rightarrow$ violation.

Step 3 — Decompose

Relation	Attributes	Key
EMPLOYEE	EmpID, EmpName, DeptID, ManagerID	EmpID
DEPARTMENT	DeptID, DeptName	DeptID

Both relations are now in 3NF — DeptName depends only on DeptID (its own key), and every other attribute depends only on EmpID. The two relations remain lossless-joinable on DeptID (the FK in EMPLOYEE).

Q5. Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

Bloom's Level: BL3 – Apply | Marks: 5

Step 1 — Find ALL candidate keys

```
closure(AB) = {A,B,C,D} -> AB is a key
closure(BC) = {A,B,C,D} -> BC is a key
closure(BD) = {A,B,C,D} -> BD is a key
closure(C)  = {A,C,D}   -> not a key (missing B)
closure(D)  = {A,D}     -> not a key (missing B,C)
```

Tool verification (Python):

```
>>> candidate_keys(attrs, FDs)
[(B, C), (B, D), (A, B)]
```

Candidate keys = {AB, BC, BD}. (B appears in every key — nothing in the FD set ever puts B on a right-hand side, so B must be a prime attribute in any key.)

Step 2 — BCNF test (every FD's LHS must be a superkey)

FD	Is LHS a superkey?	Verdict
$AB \rightarrow C$	Yes (AB is a candidate key)	OK
$C \rightarrow D$	No ($C^+ = \{A,C,D\}$, missing B)	VIOLATES BCNF
$D \rightarrow A$	No ($D^+ = \{A,D\}$, missing B,C)	VIOLATES BCNF

R is NOT in BCNF.

Step 3 — BCNF decomposition

- Use violating FD $C \rightarrow D$. C^+ (within R) = {A,C,D}. Split: $R_1(A,C,D)$ and $R_2(B,C)$.
- $R_1(A,C,D)$ still has $D \rightarrow A$ with $D^+ = \{A,D\} \neq$ all of $R_1 \rightarrow R_1$ itself is not yet in BCNF. Split again using $D \rightarrow A$: $R_{1a}(A,D)$ and $R_{1b}(C,D)$.
- $R_2(B,C)$ has only trivial dependencies $\rightarrow$ already in BCNF.

Final relation	Key	BCNF?
$R_{1a}(A, D)$	D	Yes
$R_{1b}(C, D)$	C	Yes
$R_2(B, C)$	B, C	Yes

Tool verification — sample data built in SQLite, decomposed, and rejoined:

```
sqlite> SELECT * FROM R; -- original relation
```

```
A | B | C | D
a1 | b1 | c1 | a1
a1 | b2 | c2 | a1
a2 | b1 | c3 | a2
```

```
sqlite> SELECT * FROM R1a; -- (A,D)
```

```
A | D
a1 | a1
a2 | a2
```

```

sqlite> SELECT * FROM R1b; -- (C,D)
C | D
c1 | a1
c2 | a1
c3 | a2

sqlite> SELECT * FROM R2; -- (B,C)
B | C
b1 | c1
b2 | c2
b1 | c3

sqlite> R1a NATURAL JOIN R1b NATURAL JOIN R2; -- rejoin
B | C | A | D
b1 | c1 | a1 | a1
b2 | c2 | a1 | a1
b1 | c3 | a2 | a2

```

The 3-way natural join of R1a, R1b, R2 reproduces the original R exactly (same rows, no spurious tuples) → the BCNF decomposition is lossless-join.

Q6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

Bloom's Level: BL3 – Apply | Marks: 5

Setup

Relation R(A,B,C,D) with FD $C \rightarrow D$, decomposed into R1(A,B,C) and R2(C,D).

Tabular (Chase) method

- Build a tableau with one row per decomposed relation and one column per attribute of R.
- In a row, put the shared symbol 'a<attr>' if that attribute belongs to that relation, otherwise a distinct local symbol 'b<attr><row>'.
- For every FD $X \rightarrow Y$ in the FD set: if two rows agree on all of X, force their Y-values to be equal (prefer the 'a' symbol when one exists).
- Repeat until no more symbols can be equated. If, at the end, some row consists entirely of unsubscripted 'a' symbols, the decomposition is lossless-join.

Tool verification — Python implementation of the Chase algorithm:

```

Initial tableau
  A   B   C   D
R1: aA aB aC bD1
R2: bA2 bB2 aC aD

```

```

After applying FD C -> D
  A   B   C   D
R1: aA aB aC aD
R2: bA2 bB2 aC aD

```

Result: LOSSLESS-JOIN (a full row of a-symbols found) decomposition

Row R1 became {aA, aB, aC, aD} — a full row of unsubscripted symbols — so the decomposition {A,B,C} ∝ {C,D} is confirmed LOSSLESS-JOIN.

Q7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

Bloom's Level: BL3 – Apply | Marks: 5

Step 1 — Identify the MVDs

A teacher can teach several subjects and independently pursue several hobbies — Subject and Hobby have nothing to do with each other, yet in one un-decomposed table every subject must be paired with every hobby for that teacher, causing redundancy.

MVD	Meaning
$TID \twoheadrightarrow Subject$	For a fixed TID, the set of Subjects is independent of Hobby
$TID \twoheadrightarrow Hobby$	For a fixed TID, the set of Hobbies is independent of Subject

R is in BCNF ($TID \twoheadrightarrow Subject/Hobby$ are non-trivial MVDs but there's no FD violation) yet NOT in 4NF, because a non-trivial MVD $TID \twoheadrightarrow Subject$ exists where TID is not a superkey of the whole relation.

Step 2 — Decompose to 4NF

Split R into one relation per independent multivalued fact:

Relation	Attributes
TEACHER_SUBJECT	TID, Subject
TEACHER_HOBBY	TID, Hobby

Tool verification — built and rejoined in SQLite:

```
sqlite> SELECT * FROM TEACHER; -- unnormalized (2 subjects x 2 hobbies = 4 redundant rows)
```

```
TID | Subject | Hobby
```

```
-----  
T1 | Maths | Chess  
T1 | Maths | Painting  
T1 | Physics | Chess  
T1 | Physics | Painting
```

```
sqlite> SELECT * FROM TEACHER_SUBJECT; -- 4NF table 1
```

```
TID | Subject
```

```
-----  
T1 | Maths  
T1 | Physics
```

```
sqlite> SELECT * FROM TEACHER_HOBBY; -- 4NF table 2
```

```
TID | Hobby
```

```
-----  
T1 | Chess  
T1 | Painting
```

```
sqlite> SELECT * FROM TEACHER_SUBJECT NATURAL JOIN TEACHER_HOBBY; -- rejoin reproduces TEACHER exactly => lossless
```

```
TID | Subject | Hobby
```

```
-----  
T1 | Maths | Chess  
T1 | Maths | Painting  
T1 | Physics | Chess  
T1 | Physics | Painting
```

TEACHER_SUBJECT ⋈ TEACHER_HOBBY reproduces TEACHER exactly, but each 4NF table alone now stores each fact only once — inserting a new hobby no longer requires duplicating every subject row. Redundancy is eliminated and the decomposition is lossless.

Q8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

Bloom's Level: BL3 – Apply | Marks: 5

Concept — Join Dependency (JD)

A relation satisfies a join dependency $\ast(R_1, R_2, \dots, R_n)$ if it always equals the natural join of its projections onto $R_1..R_n$. When such a JD holds and cannot be implied by any single candidate key (a genuine JD, not decomposable via BCNF alone), the relation carries redundancy that only a 3-way (or more) projection can remove — this is the domain of 5NF.

Classic example — SUPPLIER_PART_PROJECT(S, P, J)

Business rule: if supplier S supplies part P, and part P is used in project J, and supplier S supplies project J, then S must supply part P specifically for project J. This 'cyclic' rule is a genuine join dependency $\ast((S,P),(P,J),(S,J))$ — it is NOT implied by any functional dependency, so BCNF alone cannot remove its redundancy.

S	P	J
Raj Traders	Bolt	Bridge-1
Raj Traders	Nut	Bridge-1
Kumar & Co	Bolt	Bridge-1
Raj Traders	Bolt	Bridge-2

Notice the redundancy: 'Raj Traders supplies Bolt' is repeated across two project rows, and every (S,P,J) combination that is individually plausible from the pairwise facts must be re-verified — a single 3-attribute table forces this kind of duplication.

5NF decomposition (Project-Join Normal Form)

Relation	Attributes	Meaning
SP	Supplier, Part	Which parts each supplier supplies (at all)
PJ	Part, Project	Which parts each project uses
SJ	Supplier, Project	Which projects each supplier supplies to

SP $\bowtie$ PJ $\bowtie$ SJ reconstructs SUPPLIER_PART_PROJECT exactly, with each atomic fact (a supplier supplies a part; a part is used in a project; a supplier supplies to a project) stored exactly once. This is 5NF: R cannot be decomposed further without losing information, and it has no join dependency not implied by its candidate keys.

Q9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Bloom's Level: BL2 – Understand | Marks: 5

Poorly designed schema

COLLEGE_INFO(StudentID, StudentName, DeptID, DeptName, HOD, CourseID, CourseName, Marks) — everything about a student, their department, and every course they take is crammed into one flat table.

Tool verification — built in SQLite to show the anomalies live:

```
sqlite> SELECT * FROM COLLEGE_INFO; -- unnormalized single table
StudentID | StudentName | DeptID | DeptName | HOD | CourseID | CourseName | Marks
-----
S1 | Arun | D1 | CSE | Dr.Raj | C1 | DBMS | 88
```

```
S1 | Arun | D1 | CSE | Dr.Raj | C2 | OS | 75
S2 | Bala | D1 | CSE | Dr.Raj | C1 | DBMS | 91
S3 | Chitra | D2 | ECE | Dr.Kumar | C3 | Signals | 80
```

--- ANOMALY DEMONSTRATIONS ---

[Update anomaly] HOD of Dept D1 is stored redundantly in 2 rows (S1-C1, S1-C2, S2-C1).

```
sqlite> SELECT DeptID,DeptName,HOD FROM COLLEGE_INFO WHERE DeptID='D1'; -- 3 identical copies to update together
DeptID | DeptName | HOD
```

```
-----
D1 | CSE | Dr.Raj
D1 | CSE | Dr.Raj
D1 | CSE | Dr.Raj
```

[Deletion anomaly] Deleting student S3's only record erases all knowledge of Dept D2/ECE.

```
sqlite> after deleting S3 -> SELECT DISTINCT DeptID,DeptName,HOD FROM COLLEGE_INFO; -- D2/ECE/Dr.Kumar is now GONE
DeptID | DeptName | HOD
```

```
-----
D1 | CSE | Dr.Raj
```

[Insertion anomaly] Cannot add a new Department (e.g. D3-Mech) until at least one student enrolls in it (no student-independent place to store dept facts).

--- 3NF DECOMPOSITION ---

```
sqlite> SELECT * FROM DEPARTMENT; -- DeptID -> DeptName, HOD (no redundancy)
DeptID | DeptName | HOD
```

```
-----
D1 | CSE | Dr.Raj
D2 | ECE | Dr.Kumar
```

```
sqlite> SELECT * FROM STUDENT;
StudentID | StudentName | DeptID
```

```
-----
S1 | Arun | D1
S2 | Bala | D1
S3 | Chitra | D2
```

```
sqlite> SELECT * FROM COURSE;
CourseID | CourseName
```

```
-----
C1 | DBMS
C2 | OS
C3 | Signals
```

```
sqlite> SELECT * FROM ENROLLMENT;
StudentID | CourseID | Marks
```

```
-----
S1 | C1 | 88
S1 | C2 | 75
S2 | C1 | 91
S3 | C3 | 80
```

Now D2/ECE/Dr.Kumar persists in DEPARTMENT even with zero students, and HOD is stored ONCE per dept -> anomalies resolved.

Anomalies identified

Anomaly	Explanation
---------	-------------

Update	DeptName/HOD for a department is repeated on every row of every student in it — changing the HOD means updating many rows; missing even one leaves the database inconsistent.
Deletion	Deleting the only student record for a department (e.g., the sole ECE student) silently deletes all knowledge that the ECE department and its HOD exist at all.
Insertion	A brand-new department cannot be recorded until at least one student has been enrolled in it, because DeptID/DeptName/HOD have no student-independent home.

Normalize to 3NF

Functional dependencies: $\text{StudentID} \rightarrow \text{StudentName}, \text{DeptID}$; $\text{DeptID} \rightarrow \text{DeptName}, \text{HOD}$; $\text{CourseID} \rightarrow \text{CourseName}$; $\text{StudentID}, \text{CourseID} \rightarrow \text{Marks}$. DeptName/HOD depend transitively on StudentID (via DeptID) → 3NF violation; decompose:

Relation	Attributes
DEPARTMENT	DeptID, DeptName, HOD
STUDENT	StudentID, StudentName, DeptID
COURSE	CourseID, CourseName
ENROLLMENT	StudentID, CourseID, Marks

As shown in the tool output above, after decomposition DEPARTMENT rows persist independently of STUDENT rows and HOD is stored exactly once per department — all three anomaly classes are resolved.

Q10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Bloom's Level: BL3 – Apply | Marks: 5

The assessment sheet does not attach a specific relation/FD set for this question (it is normally supplied separately by the faculty). The worked method below is demonstrated on a representative relation — apply the same algorithm verbatim once the actual FDs are given.

Relation: $R(A,B,C,D,E)$ FDs: $A \rightarrow BC$, $CD \rightarrow E$

Closure algorithm

- Start result = the attribute set itself.
- Scan every FD $X \rightarrow Y$; if $X \subseteq \text{result}$ and $Y \not\subseteq \text{result}$, add Y to result.
- Repeat until one full pass adds nothing new.

```
closure(A):
  A -> BC : A ⊆ {A} → add B,C  result = {A,B,C}
  CD -> E : CD ⊄ {A,B,C} (missing D) → no change
closure(A) = {A, B, C} (NOT all attributes -> A alone is not a key)
```

```
closure(A,D):
  A -> BC : add B,C  result = {A,B,C,D}
  CD -> E : CD ⊆ result → add E  result = {A,B,C,D,E}
closure(A,D) = {A,B,C,D,E} = ALL attributes -> {A,D} IS a candidate key
```

```
closure(D):
  no FD has D (or a subset of D) alone as LHS -> closure(D) = {D}
```

Tool verification (Python attribute-closure algorithm):

```
>>> closure(["A"], FDs)    {'A','B','C'}
```

```
>>> closure(["A","D"], FDs)    {'A','B','C','D','E'}
>>> closure(["D"], FDs)        {'D'}
>>> candidate_keys(attrs, FDs)  [('A', 'D')]
```

Candidate key = {A, D}. A and D are prime attributes; B, C, E are non-prime (each derivable from the key but not part of it).

Summary

Q.No	Concept applied	Key result
1	FD identification, candidate key	Key = {SID, CID}
2	1NF conversion	Repeating group removed, key becomes {SID, CID}
3	2NF check	Already 2NF — key {A} is atomic, no partial dependency
4	3NF (transitive dependency)	Split into EMPLOYEE / DEPARTMENT
5	BCNF verification & decomposition	R1a(A,D), R1b(C,D), R2(B,C)
6	Lossless-join via tabular Chase	Confirmed lossless for {A,B,C}/{C,D}
7	MVD identification, 4NF	TEACHER_SUBJECT / TEACHER_HOBBY
8	Join dependency, 5NF	SP / PJ / SJ (classic SPJ example)
9	Anomalies, 3NF	DEPARTMENT / STUDENT / COURSE / ENROLLMENT
10	Attribute closure, candidate keys	Method demonstrated; key = {A,D} for the sample FDs